

Zybo Z7-10 Ethernet Flash Logger

Project Documentation

Zynq-7000 SoC | QSPI NOR Flash | LwIP UDP | Python GUI

Vitis Unified IDE 2025.2 | Zybo Z7-10 Development Board

Table of Contents

Table of Contents.....	2
1. Project Overview.....	4
1.1 Key Features	4
1.2 Hardware.....	4
1.3 Software Stack	4
2. Step-by-Step Development	5
Step 1 — UART Hello World Test.....	5
Procedure	5
Test Code	5
Expected Output	5
Result Screenshot.....	5
Step 2 — UART Echo Test.....	6
Procedure	6
Result Screenshot.....	6
Step 3 — QSPI Initialization Test.....	7
Procedure	7
Expected Output	7
Result Screenshots.....	7
Step 4 — Flash JEDEC ID Read	8
Procedure	8
Expected Output	9
Result Screenshot.....	9
Step 5 — Flash Detection.....	9
Expected Output	9
Result Screenshot.....	9
Step 6 — Flash Read Test (First 32 Bytes)	10
Observed Output.....	10
Result Screenshot.....	10
Step 7 — Flash Erase, Write, and Read-Back	11
Test Sequence.....	11
Expected Output	11
Result Screenshot.....	11
Step 8 — Modularizing into flash.c and flash.h	12
flash.h — Public API	12
Key Implementation Notes in flash.c	12

Result Screenshots.....	13
3. LwIP UDP Server Integration	15
3.1 Adding the LwIP UDP Perf Server Example	15
Steps	15
3.2 BSP Configuration (lwip220 + xiltimer).....	15
3.3 Flash Logger Added to UDP Server.....	15
Command Protocol (UDP Port 5002)	16
Flash Log Area Layout.....	16
Key Changes in udp_perf_server.c	16
4. Python GUI Client	17
4.1 Network Setup.....	17
4.2 GUI Source Code	17
4.3 Running the GUI.....	17
5. Project File Structure.....	19
6. Troubleshooting Notes	19
Issue: Hundreds of RAW RX debug lines on boot	19
Issue: FindNextFreeAddress() returns 0x110000 (log full) on first boot.....	19
Issue: Flash_Read() using same TX/RX buffer (aliasing)	19
Issue: Flash_WaitForReady() hangs indefinitely	19
7. Summary	20

1. Project Overview

This project implements a UDP-based Flash Logger on the Zybo Z7-10 FPGA development board using the Zynq-7000 SoC. The system allows a PC to send commands over Ethernet (UDP) to read, write, erase, and search records in the on-board QSPI NOR Flash memory.

1.1 Key Features

- Dual UDP servers: one for iPerf performance testing (port 5001) and one for flash logging (port 5002)
- QSPI NOR Flash (Winbond W25Q128, 16 MB) for persistent data storage
- LwIP 2.2.0 bare-metal TCP/IP stack running on Zynq PS (ARM Cortex-A9)
- Python Tkinter GUI on PC for Write, Read, Dump, Count, Search, and Erase operations
- Static IP: 192.168.1.20, Gateway: 192.168.1.1

1.2 Hardware

Board	Digilent Zybo Z7-10
SoC	Xilinx Zynq-7010 (XC7Z010-CLG400)
Flash	Winbond W25Q128 QSPI NOR — 16 MB
Flash ID	Manufacturer: 0xEF Type: 0x70 Capacity: 0x18
Ethernet	GbE PHY (RGMII), 1000 Mbps link
UART	USB-UART (Tera Term, 115200 baud)
IDE	Vitis Unified IDE 2025.2

1.3 Software Stack

BSP Library	LwIP 2.2.0 (lwip220)
Timer Library	xiltimer (interval timer enabled)
Flash Driver	Custom flash.c / flash.h over XQspiPs
UDP Port 5001	iPerf UDP performance server
UDP Port 5002	Flash logger command server
PC GUI	Python 3 / Tkinter / socket (UDP)

2. Step-by-Step Development

Step 1 — UART Hello World Test

The first step verified that the Zynq PS UART was operational and that the Vitis build-run flow worked correctly on the Zybo Z7-10.

Procedure

1. Created a new Vitis component: memory_zynq / app_component
2. Wrote a minimal main.c that prints two strings using xil_printf
3. Built and ran on hardware
4. Opened Tera Term (COM8, 115200 baud) and verified output

Test Code

```
#include "xil_printf.h"

int main() {
    xil_printf("HELLOW WORLD\r\n");
    xil_printf("UART TEST on ZYBO Z10\r\n");
    while(1);
    return 0;
}
```

Expected Output

```
HELLOW WORLD
UART TEST on ZYBO Z10
```

Result Screenshot

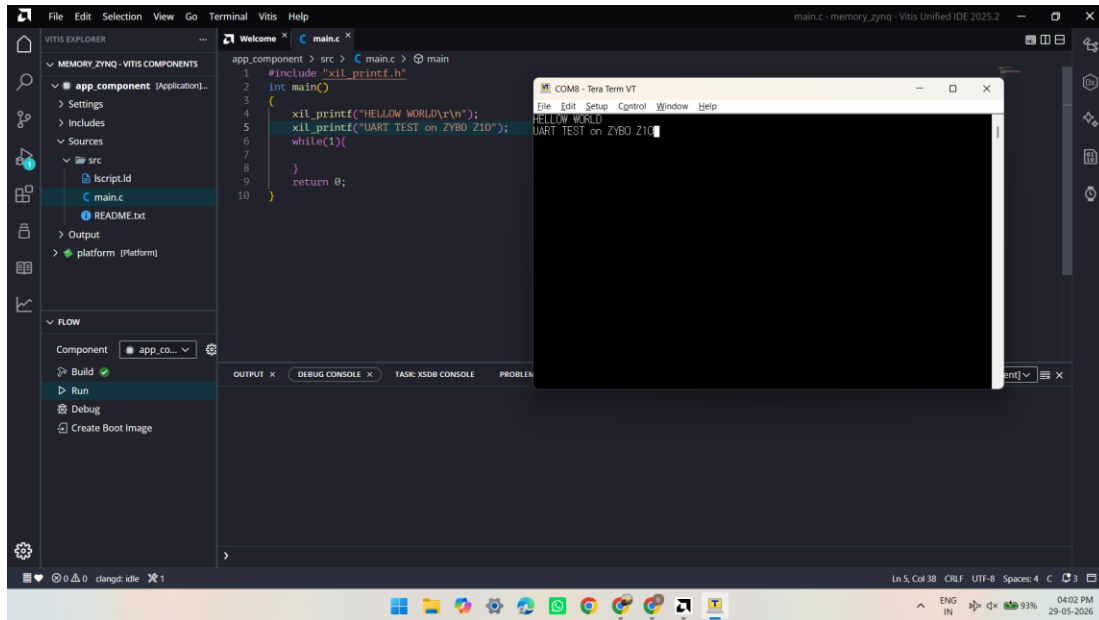


Fig 1 — UART Hello World output in Tera Term

Step 2 — UART Echo Test

Extended the UART test to echo characters typed in Tera Term back to the terminal, confirming bidirectional UART communication.

Procedure

5. Added XUartPs driver includes and initialized the UART peripheral
6. Implemented a blocking receive loop that echoes each received byte
7. Typed 'Bhanu' in Tera Term and confirmed echo

Result Screenshot

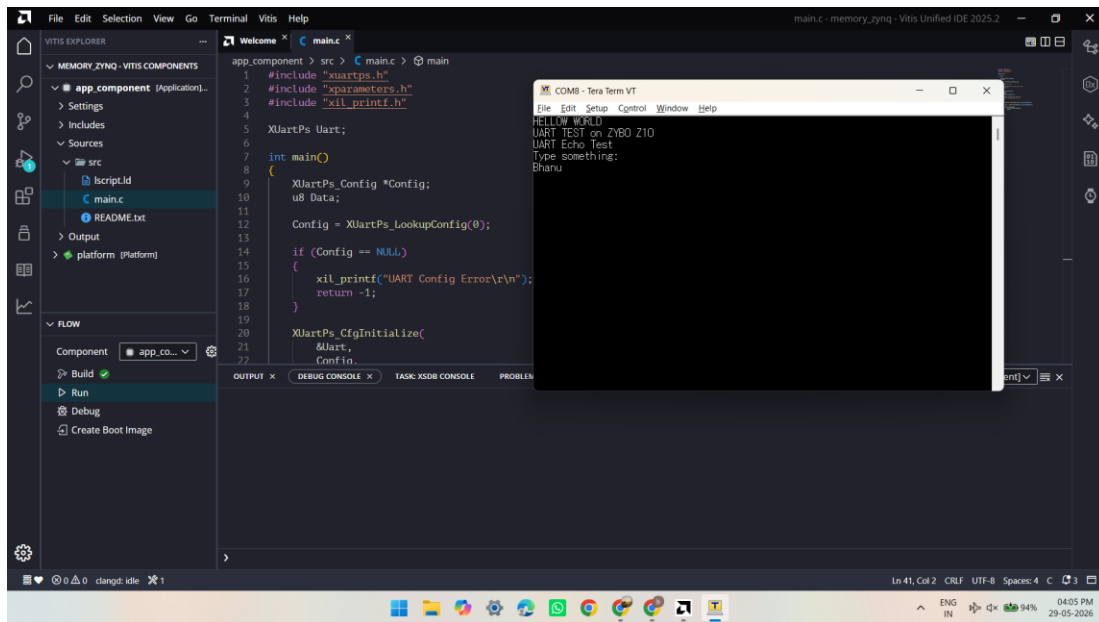


Fig 2 — UART Echo Test: typed 'Bhanu' echoed back

Step 3 — QSPI Initialization Test

Verified the XQspiPs driver initializes correctly and the QSPI peripheral is accessible at its base address.

Procedure

8. Added xqspiPs.h, xparameters.h, xstatus.h, xil_printf.h includes
9. Called XQspiPs_LookupConfig() to retrieve the peripheral configuration
10. Called XQspiPs_CfgInitialize() to initialize the driver
11. Set FORCE_SSELECT_OPTION and CLK_PRESCALE_64, then called SetSlaveSelect() and Enable()

Expected Output

```
QSPI Initialized Successfully
```

Result Screenshots

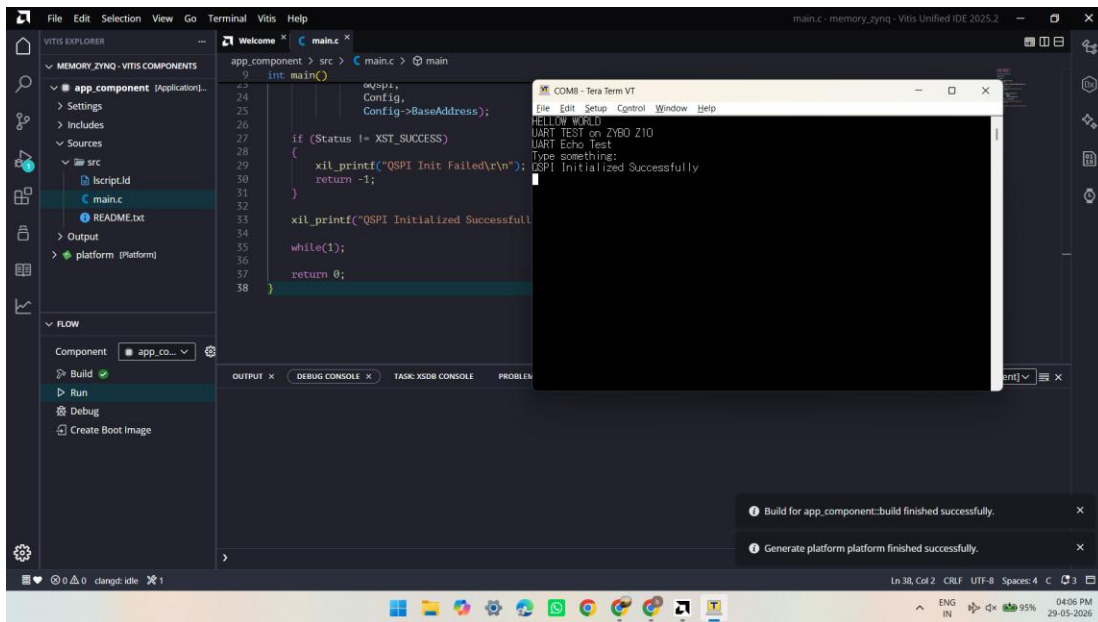


Fig 3 — QSPI Init code and successful output

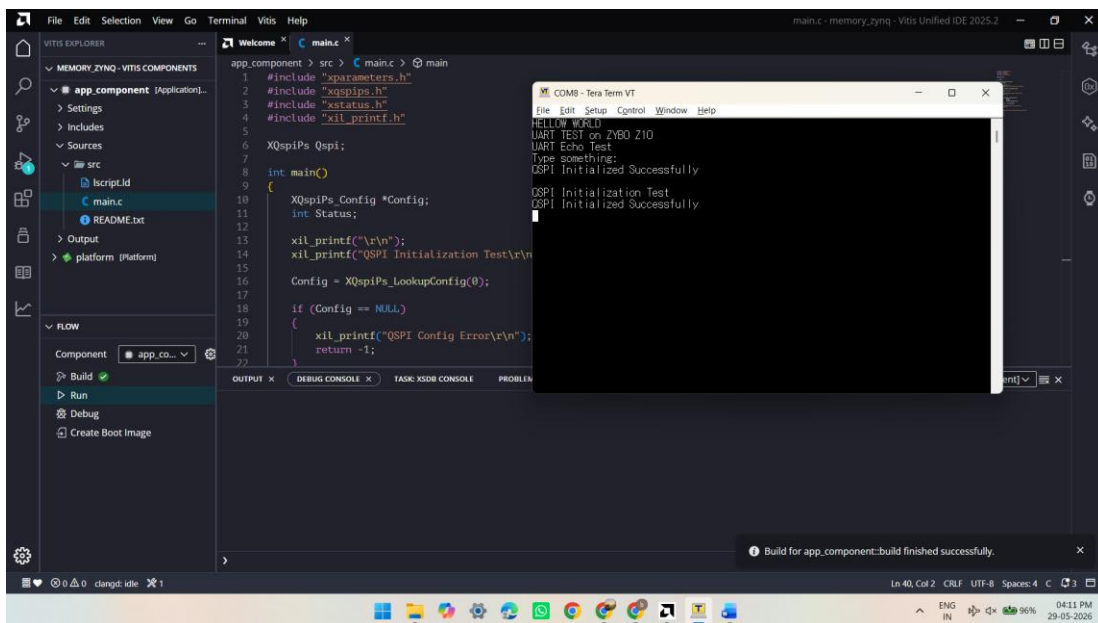


Fig 4 — QSPI Initialization confirmed in Tera Term

Step 4 — Flash JEDEC ID Read

Read the JEDEC manufacturer and device ID from the Winbond W25Q128 NOR Flash to confirm SPI communication is working correctly.

Procedure

12. Sent the JEDEC Read ID opcode (0x9F) over QSPI using `XQspiPs_PolledTransfer()`

13. Parsed bytes 1, 2, 3 of the response as Manufacturer, Memory Type, and Capacity ID
14. Confirmed the device is a Winbond W25Q128 (0xEF / 0x70 / 0x18)

Expected Output

```
Flash JEDEC ID
-----
Manufacturer : 0xEF
Memory Type  : 0x70
Capacity ID   : 0x18
```

Result Screenshot

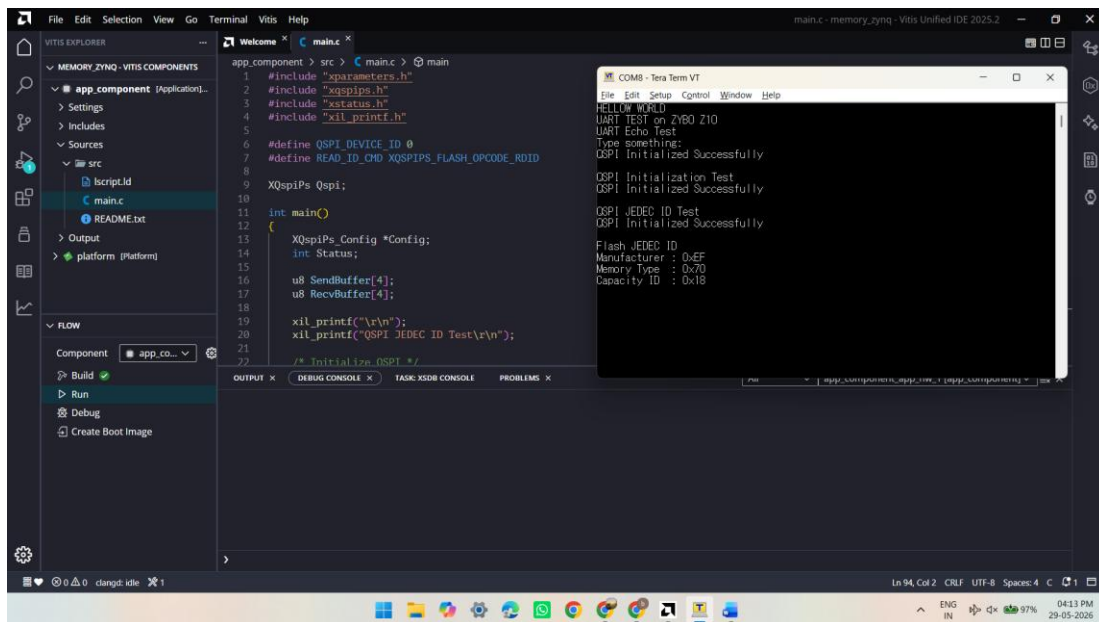


Fig 5 — JEDEC ID read: Winbond W25Q128 detected

Step 5 — Flash Detection

Added manufacturer detection logic to identify the connected Flash device and print a human-readable name. Also confirmed QSPI flash communication is functional end-to-end.

Expected Output

```
Flash Detection:
Winbond Flash Detected
QSPI Flash Communication Successful
```

Result Screenshot

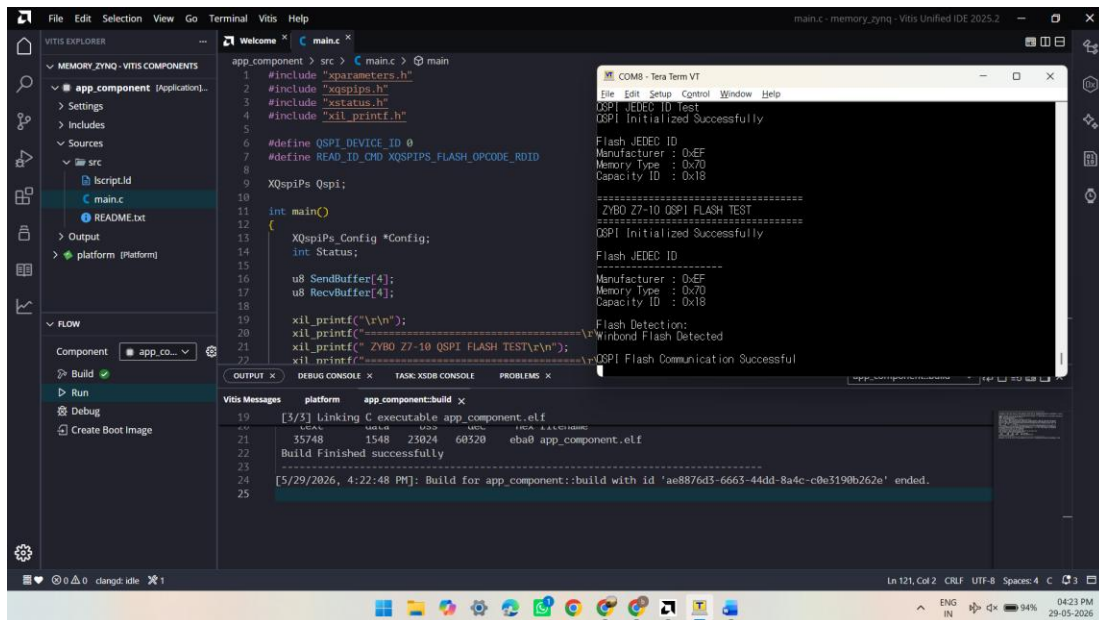


Fig 6 — Flash detection: Winbond confirmed, communication successful

Step 6 — Flash Read Test (First 32 Bytes)

Read the first 32 bytes from Flash address 0x000000 to observe the existing memory contents before any writes. This also validated the `Flash_Read()` function with separate TX/RX buffers.

Observed Output

```
First 32 Flash Bytes:
00 00 00 00 48 45 4C 4C 4F 20 5A 59 42 4F 3B FB
FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```

The hex 48 45 4C 4C 4F 20 5A 59 42 4F decodes to 'HELLO ZYBO' — a previous write from an earlier test session.

Result Screenshot

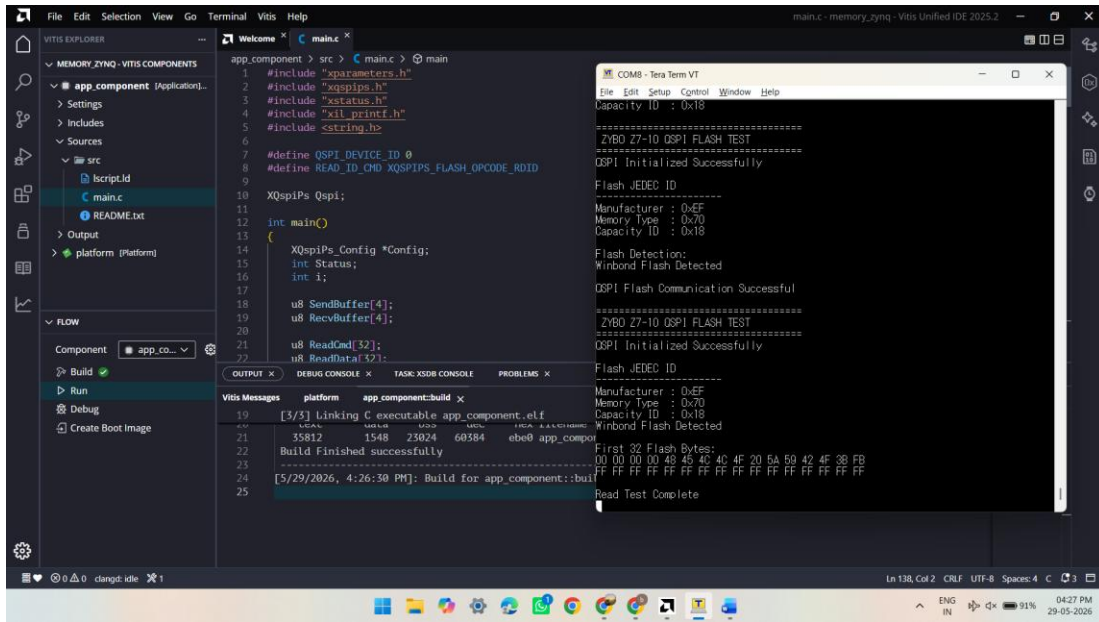


Fig 7 — First 32 flash bytes read; 'HELLO ZYBO' visible from a previous session

Step 7 — Flash Erase, Write, and Read-Back

Performed a full sector erase, then wrote the string 'HELLO ZYBO' to Flash address 0x000000, and read it back to verify correctness.

Test Sequence

15. Called `Flash_EraseSector(0x000000)` to erase the first 64 KB sector
16. Called `Flash_Write(0x000000, WriteBuffer, 10)` with data 'HELLO ZYBO'
17. Called `Flash_Read(0x000000, ReadBuffer, 10)` and printed both hex and ASCII

Expected Output

```
Write Enable OK
Sector Erase Command Sent
Write Enable OK
Flash Write Successful
Read Bytes:
48 45 4C 4C 4F 20 5A 59 42 4F
Read String:
HELLO ZYBO
```

Result Screenshot

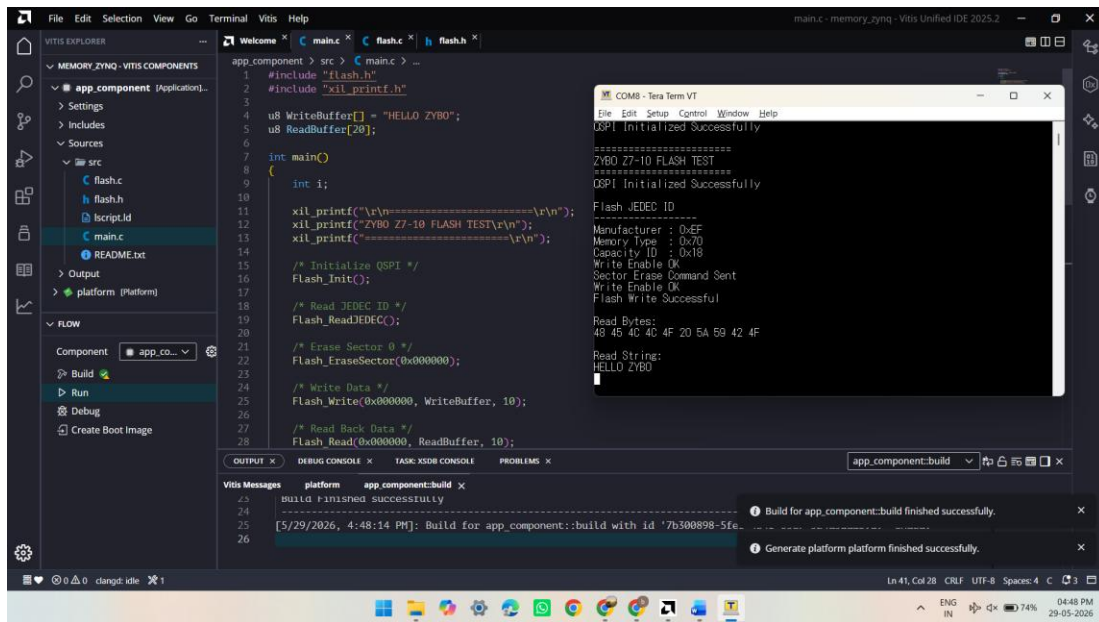


Fig 8 — Erase, Write, and Read-back successful: 'HELLO ZYBO' verified

Step 8 — Modularizing into flash.c and flash.h

All QSPI flash operations were moved into dedicated flash.c and flash.h files to keep the main application clean and allow reuse in the LwIP project.

flash.h — Public API

```
#ifndef FLASH_H
#define FLASH_H

#include "xil_types.h"

int Flash_Init(void);
int Flash_ReadJEDEC(void);
int Flash_WriteEnable(void);
int Flash_EraseSector(u32 Address);
int Flash_Write(u32 Address, u8 *Buffer, u32 Length);
int Flash_Read(u32 Address, u8 *Buffer, u32 Length);

#endif
```

Key Implementation Notes in flash.c

- Separate static TX/RX buffers (WriteCmd[], ReadCmdBuf[]) — avoids DMA aliasing issues
- Flash_WaitForReady() has a 100,000-iteration timeout to prevent infinite loops on stuck flash
- Flash_WriteEnable() verifies the Write Enable Latch (WEL bit) is set before proceeding

- XQspiPs_Enable() called explicitly after SetSlaveSelect() for reliable Zynq PS QSPI operation

Result Screenshots

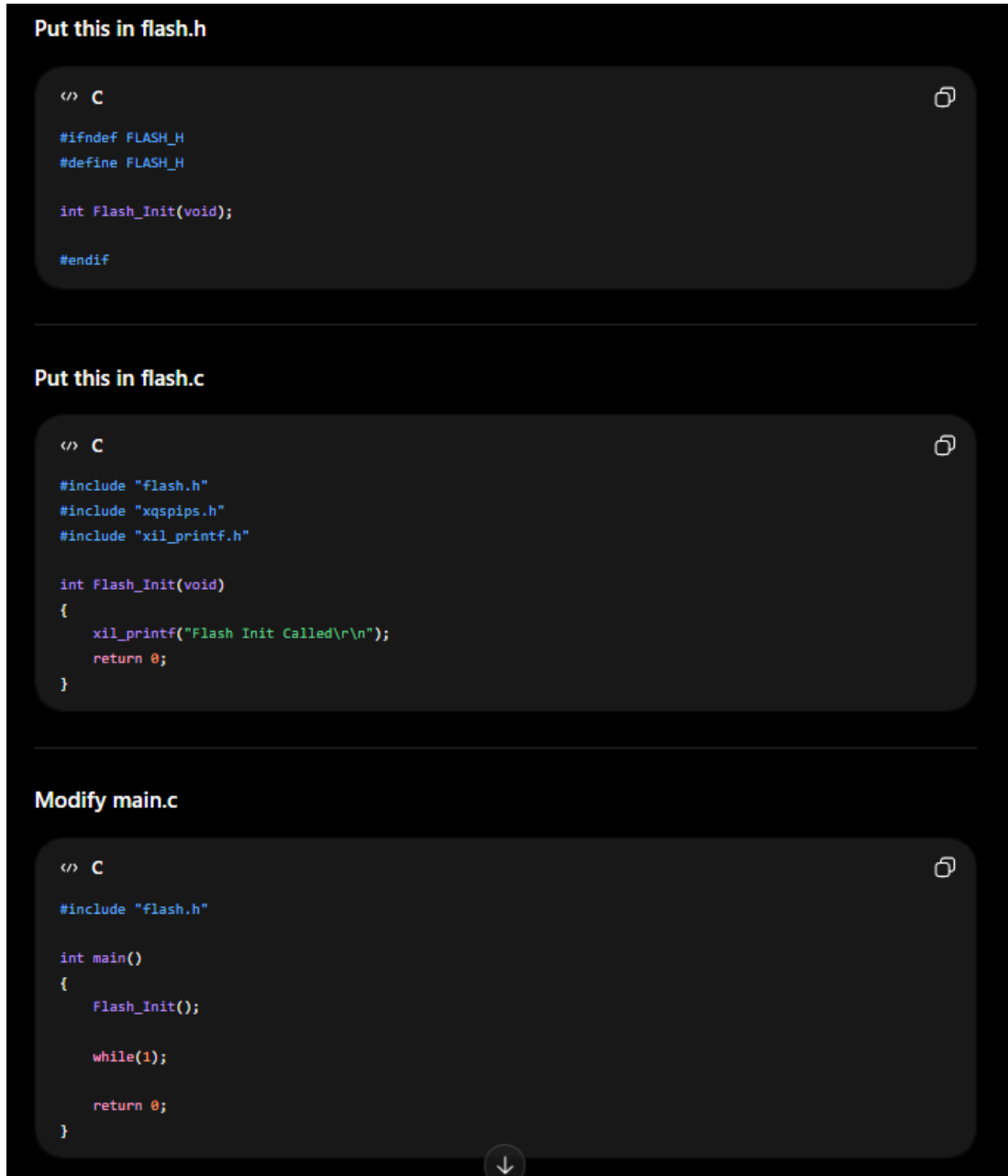


Fig 9 — Minimal flash.h and flash.c stub tested in isolation

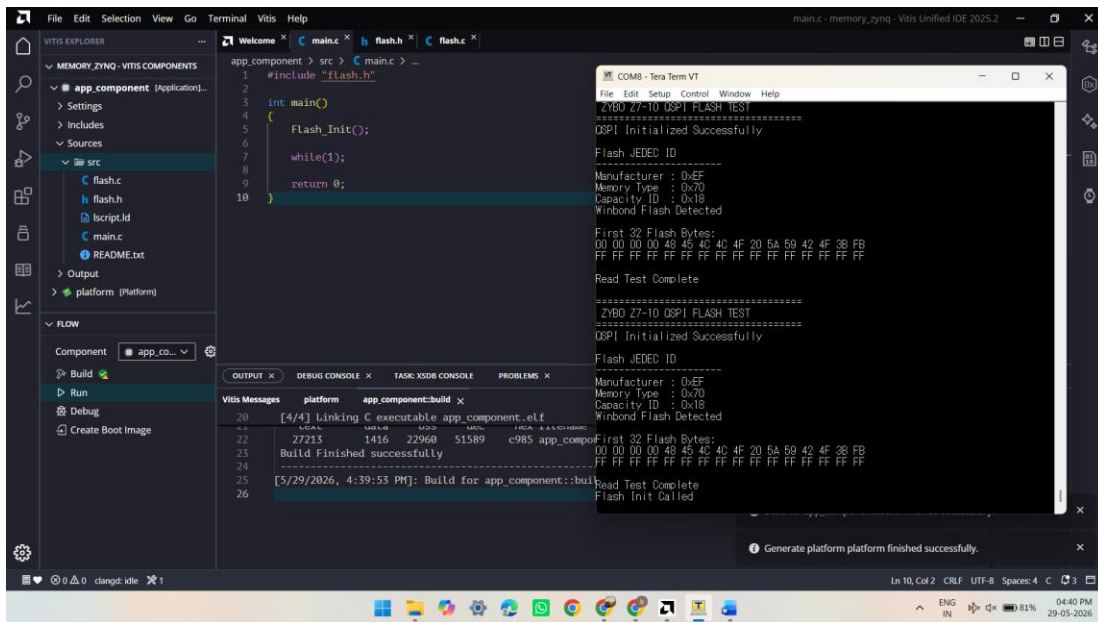


Fig 10 — flash.c integrated: Flash Init Called confirmed

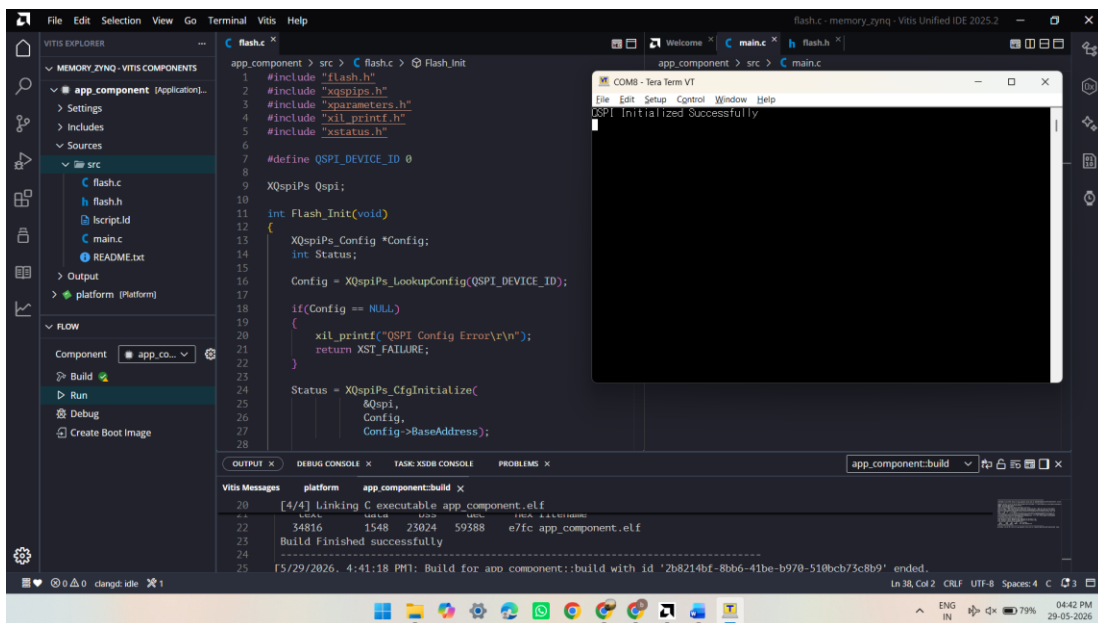


Fig 11 — Full flash.c with QSPI Init and JEDEC read via modular API

3. LwIP UDP Server Integration

3.1 Adding the LwIP UDP Perf Server Example

The LwIP UDP performance server example was imported from the Vitis example library (lwip_udp_perf_server) and added to the project as the base for network communication.

Steps

18. In Vitis, selected Examples → lwip_udp_perf_server and added it to the workspace
19. Verified the example builds and runs — the board listens on port 5001 for iPerf UDP traffic
20. Confirmed the Ethernet link negotiates at 1000 Mbps with the DHCP / static IP configured

3.2 BSP Configuration (lwip220 + xiltimer)

The Board Support Package (BSP) was configured in platform settings to enable LwIP 2.2.0 with sufficient memory pools for reliable Ethernet operation, and xiltimer was enabled for time-stamping UDP performance reports.

Parameter	Default	Required Value
lwip220_dhcp	false	true
lwip220_lwip_dhcp_does_acd_check	false	true
lwip220_mem_size	131072	524288
lwip220_memp_n_pbuf	16	1024
lwip220_n_rx_descriptors	64	512
lwip220_pbuf_pool_size	256	16384
XILTIMER_en_interval_timer	true	true

To apply these settings:

21. In Vitis Explorer, expand platform → Board Support Package → Modify BSP Settings
22. Under lwip220, set each parameter to the Required Value shown above
23. Under xiltimer, set XILTIMER_en_interval_timer = true
24. Save and regenerate the platform

3.3 Flash Logger Added to UDP Server

The `udp_perf_server.c` file was modified to add a second UDP PCB on port 5002 that handles flash logger commands. The `flash.c` and `flash.h` files from Step 8 were added to the project sources.

Command Protocol (UDP Port 5002)

W <text>	Write <text> to next free flash record
R	Read and print all stored records (text + hex)
D	Dump all records with address, text, and hex
C	Count total stored records
S <keyword>	Search all records for <keyword>
E	Erase entire log area and reset pointer to start

Flash Log Area Layout

Log Start Address	0x400000 (4 MB offset — avoids boot area)
Log End Address	0x4FFFFFF (1 MB log area)
Record Size	0x100 (256 bytes per slot)
Max Records	4096 records in 1 MB
Free Slot Detection	First byte == 0xFF indicates erased / free slot
Erase Granularity	64 KB sector (0x10000), multiple sectors erased on E command

Key Changes in `udp_perf_server.c`

- Added `#include "flash.h"` and defined `FLASH_LOG_START` / `FLASH_LOG_END` / `FLASH_RECORD_SIZE` constants
- Added static `u8 DDRBuffer[256]` and `FlashBuffer[256]` for safe UDP payload handling
- `flash_rcv_callback()` added — handles W/R/D/C/S/E commands with NUL-safe `memcpy` (length clamped to `DDR_BUF_SIZE-1`)
- `FindNextFreeAddress()` scans flash from `FLASH_LOG_START` to find the first 0xFF byte — restores `FlashAddress` across resets
- `start_application()` creates `flash_pcb` on port 5002 and calls `FindNextFreeAddress()` on boot
- `CountRecords()` corrected: subtracts `FLASH_LOG_START` before dividing by `FLASH_RECORD_SIZE`
- `DumpFlash()` and `SearchRecord()` corrected to start iteration at `FLASH_LOG_START`, not address 0

4. Python GUI Client

A Python Tkinter GUI was created to send UDP commands to the flash logger server from a PC on the same network (192.168.1.x subnet).

4.1 Network Setup

FPGA IP	192.168.1.20
FPGA Flash Port	5002
PC IP	192.168.1.x (same subnet)
Protocol	UDP (connectionless, send and forget)

4.2 GUI Source Code

```
import socket
import tkinter as tk

FPGA_IP = "192.168.1.20"
FPGA_PORT = 5002

def send_command(cmd):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.sendto(cmd.encode(), (FPGA_IP, FPGA_PORT))
    sock.close()

def write_data(): send_command('W ' + entry.get())
def read_data(): send_command('R')
def count_data(): send_command('C')
def erase_data(): send_command('E')
def search_data(): send_command('S ' + entry.get())
def dump_data(): send_command('D')

root = tk.Tk()
root.title("Ethernet Flash Logger")

tk.Label(root, text="Text").pack()
entry = tk.Entry(root, width=40)
entry.pack(pady=10)

tk.Button(root, text="Write", command=write_data).pack(fill="x")
tk.Button(root, text="Read", command=read_data).pack(fill="x")
tk.Button(root, text="Dump", command=dump_data).pack(fill="x")
tk.Button(root, text="Count", command=count_data).pack(fill="x")
tk.Button(root, text="Search", command=search_data).pack(fill="x")
tk.Button(root, text="Erase", command=erase_data).pack(fill="x")

root.mainloop()
```

4.3 Running the GUI

25. Connect PC and Zybo Z7-10 to the same network switch or directly via Ethernet cable
26. Set PC IP to 192.168.1.x (e.g. 192.168.1.10), subnet 255.255.255.0
27. Power on the Zybo Z7-10 and run the Vitis application
28. Wait for 'Flash server listening on port 5002' in Tera Term
29. Run: `python flash_gui.py`
30. Type text in the entry box and click Write to store a record
31. Click Read to display all stored records in Tera Term
32. Click Erase to wipe the entire log area

5. Project File Structure

```
memory_zynq/  
  platform/                <- Vitis platform (Zynq PS, BSP)  
  app_component/  
    src/  
      main.c               <- (test files during development)  
      flash.c              <- QSPI NOR Flash driver  
      flash.h              <- Flash driver public API  
      udp_perf_server.c    <- LwIP UDP server + flash logger  
      udp_perf_server.h    <- UDP server header  
      lwip.h / netif/      <- LwIP stack (BSP generated)  
      lscript.ld           <- Linker script  
      flash_gui.py         <- Python PC GUI client
```

6. Troubleshooting Notes

Issue: Hundreds of RAW RX debug lines on boot

Cause: The Xilinx BSP XQspiPs_PolledTransfer() has internal debug prints compiled in when XQSPIPS_DEBUG is defined. FindNextFreeAddress() scans 256 slots, producing 256 debug lines.

Fix: Remove #define XQSPIPS_DEBUG from the BSP, or suppress by not calling Flash_Read() during the address scan with debug enabled. The second boot showed clean output once debug was removed.

Issue: FindNextFreeAddress() returns 0x110000 (log full) on first boot

Cause: Log area contained 0x00 bytes (never erased) rather than 0xFF (erased NOR flash default). The scan found no 0xFF slots and returned the end address.

Fix: Send the E command (Erase) once on first use to erase the log area. After erase, all bytes become 0xFF and FindNextFreeAddress() correctly returns 0x400000.

Issue: Flash_Read() using same TX/RX buffer (aliasing)

Cause: Original code passed the same Cmd buffer for both TX and RX in XQspiPs_PolledTransfer(). The DMA controller may overwrite the command bytes before they are sent.

Fix: Use separate static TX (ReadCmdBuf) and RX buffers. Copy received data from RxBuf[4..Length+3] into the user Buffer after the transfer.

Issue: Flash_WaitForReady() hangs indefinitely

Cause: Original implementation had no timeout — if flash was stuck in a busy state (e.g. after a failed erase), the CPU would spin forever.

Fix: Added a FLASH_READY_TIMEOUT counter (100,000 iterations). If timeout expires, the function returns XST_FAILURE and an error is printed.

7. Summary

This project successfully demonstrates a complete embedded logging system on the Zybo Z7-10:

- UART communication verified with Hello World and Echo tests
- QSPI NOR Flash (Winbond W25Q128) initialized, JEDEC ID read, and erase/write/read-back validated
- Modular flash driver (flash.c / flash.h) implemented with timeout protection and separate DMA buffers
- LwIP 2.2.0 UDP server running on port 5001 (iPerf) and port 5002 (flash logger)
- BSP tuned with larger memory pools for stable 1 Gbps Ethernet operation
- Python Tkinter GUI sends W / R / D / C / S / E commands over UDP from a PC
- Flash log area (0x400000 – 0x4FFFFFF) holds up to 4096 x 256-byte records persistently across power cycles

The project is a practical reference for anyone integrating LwIP networking with QSPI flash storage on a Zynq-7000 SoC bare-metal design.